

The Causal Graph CRDT for Complex Document Structure

Aaron Hall
aaron.hall@workiva.com
Workiva, Inc.
2900 University Blvd
Ames, IA 50010, U.S.A.

Mike Thiesen
mike.thiesen@workiva.com
Workiva, Inc.
2900 University Blvd
Ames, IA 50010, U.S.A.

Grant Nelson
grant.nelson@workiva.com
Workiva, Inc.
2900 University Blvd
Ames, IA 50010, U.S.A.

Nate Woods
nate.woods@workiva.com
Workiva, Inc.
2900 University Blvd
Ames, IA 50010, U.S.A.

ABSTRACT

Commutative Replicated Data Types (CRDTs) are an emerging tool for real-time collaborative editing. Existing work on CRDTs mostly focuses on documents as a list of text content, but large documents (having over 7,000 pages) with complex sectional structure need higher-level organization. We introduce the Causal Graph, which extends the Causal Tree CRDT into a graph of nodes and transitions to represent ordered trees. This data structure is useful in driving document outlines for large collaborative documents, resolving structures with over 100,000 sections in less than a second.

CCS CONCEPTS

•Information systems → Document representation; *Data structures*; •Applied computing → Document management and text processing;

ACM Reference format:

Aaron Hall, Grant Nelson, Mike Thiesen, and Nate Woods. 2018. The Causal Graph CRDT for Complex Document Structure. In *Proceedings of ACM Symposium on Document Engineering 2018, Halifax, NS, Canada, August 28–31, 2018 (DocEng '18)*, 4 pages. DOI: 10.1145/3209280.3229110

1 INTRODUCTION

Collaboratively editing documents among multiple sites and users across the Internet is becoming an important tool for sharing information and coordinating people. But implementing tools to support it raises problems of maintaining accuracy and authorship while also providing interactivity and efficiency to users.

Simple methods like last-editor-wins or partial locking have been used with projects like Wikipedia or revision control systems, but those methods have problems maintaining performance and reliability and don't scale well. A more sophisticated approach uses

operational transformation (OT) [2] to mutate edits in a way that preserves causality and convergence properties despite inconsistent ordering. However, OT is often difficult to implement correctly against subtle cases and retains performance problems in scaling.

A “commutative replicated data type” (CRDT) has several advantages for real-time collaborative distributed document editing, such as consistency, [6] efficiency, [1] interactivity, [7] and respect for user intention. [11] A CRDT is a replicated data structure that is inherently commutative: it allows edits to be applied to it in any order to produce the same result. Most discussion of CRDTs involves a list-of-atoms abstract data type; when used to represent text these atoms could be single characters or entire lines of plain text. While there is work on richer CRDT structures like a JSON analogue [5] or the Directed Graph CRDT [8]; in the application to higher-level document structure, like formatting or sectioning, this is left to the application consuming the list ADT through mechanisms like control codes embedded in the atom list.

To more naturally provide the benefits of optimistic distributed editing at the higher structural level of document sections and outlines, we describe the “causal graph” data structure:¹ an ordered tree ADT suitable for representing the hierarchical structure of a document outline and related components. For use in a document, the textual components themselves can be implemented as another type of CRDT, the Causal Tree. [3, 9, 10] The Causal Graph is itself an extension of the Causal Tree CRDT: just as the Causal Tree uses an internal tree to implement a sequence CRDT, the Causal Graph uses an internal digraph to implement a hierarchy-of-sequences (ordered tree) CRDT. This combination creates an effective foundation for collaborative editing large documents, around 7,000 pages in size with 1,000 of sections and 1,200 embedded objects.

2 DESIGN OF THE CAUSAL GRAPH

In essence a Causal Graph is an ensemble of causal subtrees, each one representing the ordered parent/child relationship between levels of nodes in an ordered tree hierarchy. Taken together this forms a digraph where the edges are causal connections and vertices are node and transition objects. Moving content from one location to another in the ordered tree involves inserting a new transition in the appropriate Causal Tree.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DocEng '18, Halifax, NS, Canada

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
978-1-4503-5769-2/18/08...\$15.00
DOI: 10.1145/3209280.3229110

¹patent pending

The Causal Graph is append-only, with all mutating operations adding new vertices to the graph. In this way the graph contains a record of all its historical states. Producing an ordered tree from the Causal Graph involves a traversal of the structure, iterating through selected edges to determine which nodes are “active” (not deleted) and their hierarchy and order positioning in the current document state.

2.1 Background: Causal Trees

A Causal Tree [3, 4] is a sequence CRDT designed to represent an eventually consistent list of atoms. The list of atoms is represented by a tree of vertices. Edits carry the changes in vertices from one site with a replica of the data structure to another. When a site receives edits from another site, the edits are performed. Regardless of the order the edits are performed in, consistency will eventually be reached.

Each vertex in a Causal Tree must have a globally unique id, written $\#s:t$, where s is an integer identifying a site, and t is a logical timestamp that monotonically increases at each site. The site identifier is unique per site at any moment such that each site will produce unique ids. The site ids can be tied to additional information to represent authorship. When edits from another site is received the the timestamp for the site will be updated to the maximum timestamp. Vertex ids have a global “younger” order where larger timestamps are younger, with ties resolved by the site id. Formally, if a and b are vertex ids, $a > b \equiv a.t > b.t \vee (a.t = b.t \wedge a.s > b.s)$.

Each vertex in the tree contains a list of children sorted by the identifiers, youngest to oldest. These children were “caused” by the parent vertex. Each edit carries the identifier of the new vertex and the cause identifier so that the new vertex gets added to the correct list. If a cause can not be found the edit is pending until the cause has been added to the data structure.

2.2 Causal Graph Nodes and Transitions

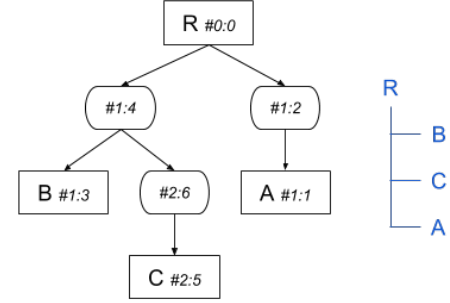
Unlike a Causal Tree, where the structural relationship among data atoms is maintained entirely in the causal edges between graph vertices, a Causal Graph differentiates between node and transition vertices, with causal edges connecting both types, and a more complex graph topology. This distinction allows for the original node to stay within the graph be moved as many times as necessary by adding transitions to it. The structures of these vertex types are shown in Table 1 (along with a few additional vertex types that will be described later).

In Figure 1, vertices $R\#0:0$, $A\#1:1$, $B\#1:3$, and $C\#2:5$ are nodes representing atoms of the Ordered Tree. Vertices $\#1:2$, $\#1:4$, and $\#2:6$ are transitions expressing the structural relationship among the nodes. The directed edges between vertices depict the causal relationships among the nodes and transitions.

2.3 Traversal as an Ordered Tree

The causal graph is traversed from its root node R to produce an ordered tree view of the relationship among its nodes. A sketch of the procedure is shown in Algorithm 1. First in the upward stage the most recent active transitions for each node are identified, then in the downward stage the traversal proceeds from R in a depth-first and inorder fashion, with sibling vertices being visited

Figure 1: A simple causal graph along with its corresponding ordered tree interpretation.



Vertex Type	Structure	Cause Can Be
node	(id, cause, atom)	transition
transition	(id, cause, start, end)	node, transition
deletion	(id, cause)	any
format	(id, cause, key, value)	node

Table 1: Vertex types in a Causal Graph.

according to the causal id younger→older order described above. The visit order and nesting depth of the nodes yields the complete ordered tree structure of the most recent document state. Since all previous state data is retained in the graph, older Ordered Tree states can be produced by selecting a timestamp $t_x < t_M$, where t_M is the maximum timestamp in the graph, and traversing only those vertices with a stamp less than or equal to t_x .

In Figure 1, the traversal sees nodes B , C , and A in that order and at the same hierarchical level, producing the ordered tree depicted at the right. Transition $\#1:4$ is visited before $\#1:2$ due to having the younger id. In the graph of Figure 2, the traversal visits B via transition $\#1:4$ and would go on to reach C (placing it under B in the ordered tree) had transition $\#1:7$ not been deleted. Instead the next node visited is A , followed by C , resulting in the ordered tree depicted.

2.4 Basic Operations

The basic operations on a Causal Graph are to *insert*, *delete*, and *move* a node.

Insert(atom, cause, after) creates both a node vertex to store the data and a transition vertex to locate it in the structure. These vertices are linked via causal edges into the existing graph under the operation’s cause.

In the graph of figure 1, site 1 inserts atom B under node $R\#0:0$ at the beginning of the child list by invoking *insert*($B, R\#0:0, null$), which creates node $B\#1:3$ and transition $\#1:4$ and appends them to the graph with the shown causal edges. Similarly, site 2 inserts C under $R\#0:0$ and after $B\#1:3$ (*insert*($C, R\#0:0, B\#1:3$)), appending $C\#2:5$ and $\#2:6$. Note the positioning requires site 2 to know about site 1’s insertion of B . The insertion finds the transition controlling $B\#1:3$ ’s position (in this case, $\#1:4$), to be the cause for transition $\#2:6$ controlling C ’s node.

Data: G : a causal graph
Result: T : an ordered tree

```
// (1) Create a Flat-tree Node List
let  $F$  be a list of (node, transition) pairs, where
 $(n, \text{null}) \in F \iff n \in G.\text{nodes}$ 
for  $n \in G.\text{nodes}$  do
  let  $P$  be  $\{p_i \in G.\text{transitions} \mid p_i.\text{end} = n\}$ 
  let  $p_m$  be the youngest active  $p_i \in P$ 
  add  $(n, p_m)$  to  $F$ 
  (later: add cycle resolution)
end
// (2) Traverse Tree
func visit( $v$  : vertex) do
  let  $C$  be the child transitions of  $v$  ordered by youngest ( $>$ )
  for  $c \in C$  do
    if  $(n, c) \in F$  for some  $n \wedge n$  is active then
      append  $n$  to  $T$ 
      visit( $n$ )
    end
  end
  visit( $c$ )
end
end
append  $G.\text{root}$  to  $T$ 
visit( $G.\text{root}$ )
```

Algorithm 1: Traversal as an Ordered Tree

Delete(target) inserts a deletion vertex (a tombstone) causally linked to the target. All vertex types—including other deletions—can be deleted. In figure 2 site 1 has deleted the transition moving $C\#2:5$ under $B\#1:3$ by inserting $\Diamond\#2:8$ under it.

Move(target, cause, after) could delete the existing node and insert a new one, but two or more simultaneous moves could unnecessarily duplicate large branch copies and yield an undesirable tree shape. Instead, *move* inserts a newer transition vertex representing the new location for the target node. During ordered-tree traversal of the Causal Graph the newer transition supersedes any older ones, respecting users' edit intentions. In figure 2 if the deletion $\Diamond\#2:8$ did not exist, the transition $\#1:7$ would be newer than the transition $\#2:6$, and atom C would be located under B in the ordered tree. As it is, because of the deletion, C is located under A .

2.5 Cycle Resolution

In practice, the nature of collaborative editing means it's possible for edits to take place "simultaneously" as far the converged Causal Graph state can tell. Consider the situation in Figure 3. Site 1 moves node A under node B at timestamp 5, while site 2 simultaneously moves node B under node A at the same timestamp. Once all edits propagate to all replicas of the data structure, how should nodes A and B be positioned in the ordered tree consistently?

An effective resolution to these cycles in the graph will produce an ordered tree that preserves commutativity while ideally also respecting user intentions. Our approach is to modify step (1) of the previous traversal algorithm, performing Algorithm 2 at the point marked *add cycle resolution*.

Figure 2: Hierarchical nesting of nodes, moving a node, and a deletion of a transition

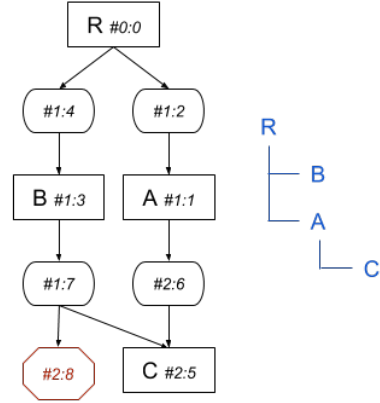
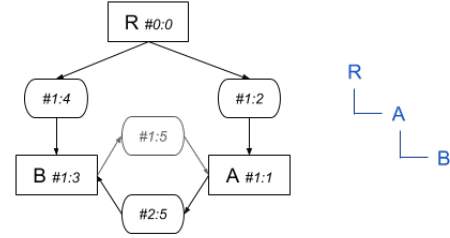


Figure 3: Transition cycle created by conflicting simultaneous move edits.



While setting the transitions in the Flat-tree Node List (FNL), if inserting a t_m leads to the formation of a cycle, the *oldest* transition in the cycle is replaced with the next-older transition not causing a cycle. If no such transition is available, the next-oldest transition in the cycle is checked for a replacement, and so on. The search is guaranteed to terminate as the FNL has no cycles before inserting each transition, and ultimately any new transition could be rejected to preserve that. By disregarding older transitions to resolve cycles, user intentions are better respected as new *move* operations are appended.

A slightly more involved situation is shown in Figure 4. Prior to timestamp 7 the state of the graph would produce an ordered tree ($R(CBA)$). At timestamp 7 each of three sites moves a different node under some other node—prior to communicating their edits, they see three different ordered trees: 1 sees ($R(B(C)A)$), 2 sees ($R(C(A)B)$), while 3 sees ($R(CA(B))$). Once those edits are exchanged among all replicas the traversal produces a single consistent ordered tree by rejecting transition $\#3:7$ in stage (1) in favor of transition $\#1:4$. Then stage (2) yields the ordered tree ($R(B(C(A)))$) shown at the right in Figure 4.

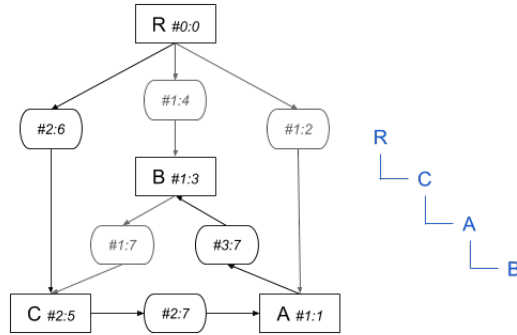
2.6 More Operations

To be a practical data structure the Causal Graph supports some additional operations: *format* and *undo/redo*. A *format(target, key,*

Data: F a list of (node,transition) pairs
Data: (n, p_m) the latest pair added to F
Result: F after modifications to resolve cycles
repeat
 $F \leftarrow \text{loopresolve}(F)$
until F does not change
func $\text{loopresolve}(F)$ **do**
 let L be a list of transitions, initially empty
 let $p = p_m$
 while $p \notin L$ **do**
 if p is null $\vee p.\text{deleted}$ **then**
 // no cycle created by r in F
 return
 end
 append p to L
 $p \leftarrow p'$ where $(p.\text{start}, p') \in F$
 end
 // cycle found, attempt to resolve
 for $l \in L$ ordered by older ($<$) **do**
 let R be all transitions $\{r \in F \mid r.\text{end} = l.\text{end}\}$
 for $r \in R$ ordered by older ($<$) **do**
 if $r > l \wedge \neg r.\text{deleted}$ **then**
 replace (n, l) with (n, r) in F
 return
 end
 end
 end
end

Algorithm 2: Cycle resolution in graph traversal.

Figure 4: A more elaborate cycle caused by transitive simultaneous moves.



value) operation involves another vertex type format which associates key/value metadata with nodes. They represent various properties that can be set on an element such as a document section name or visibility. Format vertices can be causally connected to nodes and can be deleted. The newest non-deleted format in a set of formats with the same key on the same node will be active and the rest inactive.

While $\text{undo}(\text{target})$ and $\text{redo}(\text{target})$ are treated as distinct operations a user may perform, in the data structure they're implemented as delete vertices on whatever edit vertex was undone or redone. This works as an undo of a deletion of a node appears as a deletion of that deletion, rendering the node undeleted. Formats and deletions can be deleted and undeleted in the same way.

3 AS A DOCUMENT OUTLINE

Using this Causal Graph to support a document outline is straightforward: each node vertex represents a document section, with a format vertex storing the unique key identifying its section contents. Other formats store section titles, formatting properties like page margins or numbering style, etc. The ordered-tree traversal described above produces an outline structure for the current state of the document. During our implementation and testing of Causal Graphs, we were able to produced flattened outlines for graphs containing 100k nodes in less than a second.

4 CONCLUSIONS

In this paper we have described the Causal Graph CRDT. It extends the Causal Tree CRDT by introducing a distinction between node and transition vertex types, and allowing causal edges to form a digraph topology instead of a tree. Thus it represents an ordered tree ADT (a hierarchy of sequences) and allows nodes to be moved without the overhead and possible unexpected content duplication of a delete + copy as in a Causal Tree. The intent is to combine this with Causal Tree text representation to effectively support large documents with complex section and outline structures.

In the future we plan to further investigate applications of the Causal Graph CRDT to document structure and other collaborative editing situations.

REFERENCES

- [1] Mehdi Ahmed-Nacer, Claudia-Lavinia Ignat, Gérald Oster, Hyun-Gul Roh, and Pascal Urso. 2011. Evaluating crdts for real-time document editing. In *Proceedings of the 11th ACM symposium on Document engineering*. ACM, 103–112.
- [2] Clarence A Ellis and Simon J Gibbs. 1989. Concurrency control in groupware systems. In *Acm Sigmod Record*, Vol. 18. ACM, 399–407.
- [3] Victor Grishchenko. Causal trees: towards real-time read-write hypertext. (????).
- [4] Victor Grishchenko. 2010. Deep hypertext with embedded revision control implemented in regular expressions. In *Proceedings of the 6th International Symposium on Wikis and Open Collaboration*. ACM, 3.
- [5] Martin Kleppmann and Alastair R Beresford. 2017. A Conflict-Free Replicated JSON Datatype. *IEEE Transactions on Parallel and Distributed Systems* 28, 10 (2017), 2733–2746.
- [6] Marc Shapiro—Nuno Preguiça. 2007. Designing a commutative replicated data type. *arXiv preprint arXiv:0710.1784* (2007).
- [7] Hyun-Gul Roh, Jin-Soo Kim, Joonwon Lee, and Seungryoul Maeng. 2009. *Optimistic operations for replicated abstract data types*. Technical Report. Technical Report CS/TR-2009-318. KAIST.
- [8] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems*. Springer, 386–400.
- [9] Michael J. Thiesen, Dominic J. Frost, Stephen Bush, Grant Nelson, and Benjamin Echols. 2015. System and method for convergent document collaboration. (2015). U.S. Patent 9,292,482.
- [10] Michael J. Thiesen, Dominic J. Frost, Stephen Bush, Grant Nelson, and Benjamin Echols. 2017. System and method for convergent document collaboration. (2017). U.S. Patent 9,552,343.
- [11] Stéphane Weiss, Pascal Urso, and Pascal Molli. 2009. Logoot: A scalable optimistic replication algorithm for collaborative editing on p2p networks. In *Distributed Computing Systems, 2009. ICDCS'09. 29th IEEE International Conference on*. IEEE, 404–412.